

THE BYTECODE BOUNCER

DEADLINE	MARKS
13 NOV · 11:59 PM IST	100

THE STORY

It is a dark and stormy Saturday night at Pixel Arcade, the city's most legendary LAN gaming café (legendary mostly for its samosas). Forty gamers, sixty energy drinks and one very tired game server, 10.0.0.1, which has been lagging all week like a dial-up modem in a thunderstorm.

Two villains are on the loose. One keeps spamming the old game-sync service on UDP ports 6000–6099 with fake high scores, so their name sits on top of every leaderboard. The other pings the server so often that it has started to feel, well, a little pinged.

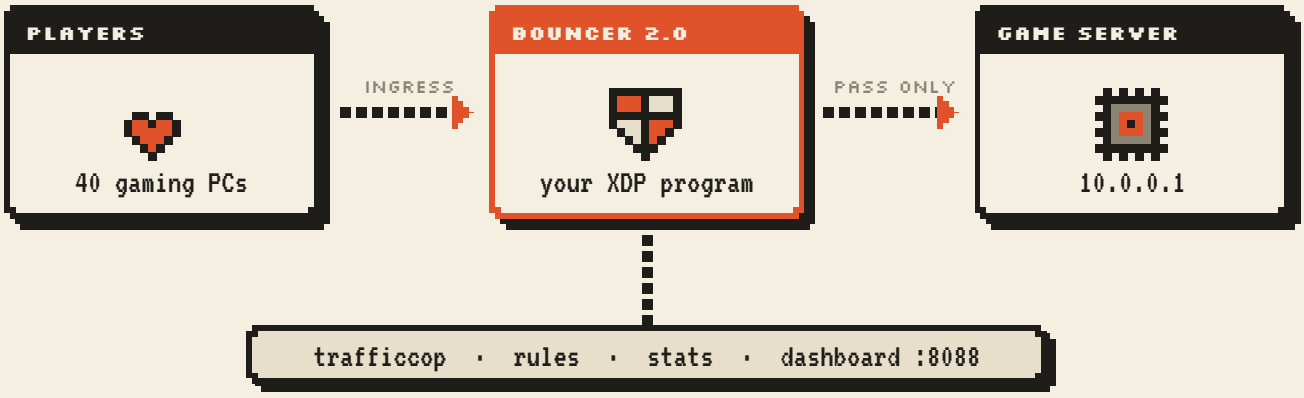
Last month Rohan, the part-time admin and full-time chai enthusiast, wrote an XDP program called the bouncer to guard the door. It let cheaters stroll in through a VLAN-shaped side entrance, mistook IP options for port numbers, and turned away innocent players because their addresses happened to end in the same digits. Then Rohan vanished into exam season, leaving behind a folder of very important evidence.

The owner has had enough. She is hiring your team to build Bouncer 2.0: a real firewall that runs in the kernel at the very edge of the network card, takes its orders from a rules file, keeps the ping-spammers in check, throws repeat offenders in jail, catches port scanners, bounces cheaters straight back out of the door, lures the sneakiest into a honeypot, speaks IPv6 as well as IPv4, writes down everything it sees, and can change the rules mid-tournament without anyone noticing a thing.

Build it well, and Pixel Arcade gets its game back. Build it badly, and you will be the next sticky note.

THE MISSION

Build **trafficcop**: an eBPF/XDP firewall for a Linux gateway, with a small control program around it. Every packet decision happens inside XDP, before the kernel's network stack ever sees the frame.



#	FEATURE	IN ONE LINE
1	Rule engine	Reads a rules file and decides PASS or DROP for every frame, first match wins
2	Rate limiter	Per-source token buckets, so each player gets a fair share and spammers don't
3	Counters	Exact counts of everything it passed, dropped and why, readable as JSON
4	Dashboard	A live web page showing traffic, rule hits and the noisiest players
5	Hot reload	Swap in a new rules file while traffic flows, without dropping a single good packet
6	Clean lifecycle	Starts fast, rejects bad rule files safely, and leaves nothing behind when stopped
7	Address sets	Named lists of networks in the rules file, editable live without a reload
8	Jail	Locks a source out completely for a while, with commands to list and release prisoners
9	Port-scan detector	Spots a source probing too many ports too fast, and jails it
10	Bounce	Sends a cheater's frame straight back out of the door it came in
11	Event log	Streams every decision of chosen rules to user space, losing nothing
12	IPv6	Every feature works for IPv6 too, extension headers and all
13	Honeypot	Quietly diverts chosen traffic to a second interface instead of the server

HOW YOU ARE JUDGED

Your score depends on two things: automatic tests of your build, and a viva where the judges talk to your team about it.

For the automatic part, a harness creates a test network, starts your trafficcop on it, loads rule files you have never seen, fires traffic you have never seen through it, and checks every verdict, counter and timing against this spec. That part is not marked by hand, so the spec on the next pages is the whole contract. Follow it exactly.

ROHAN'S FOLDER

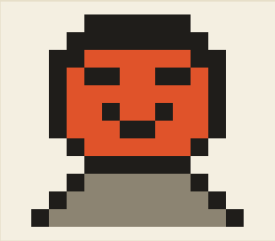
Everything Rohan left behind, as found. Just for fun: nothing on these pages changes the spec.

EXHIBIT 1 · ROHAN'S APOLOGY

Things I got wrong in v1 - Rohan
- skipped anything with a VLAN tag
- assumed every IP header is 20 bytes
- counted pings by LAST OCTET only
- my 'fix' never got past the verifier
- forgot fragments have no ports
sorry!! do better than me :)

EXHIBIT 2 · WANTED

WANTED



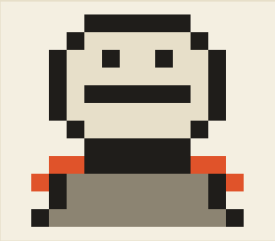
THE SCORE FORGER

xX_N00bSlayer_Xx

Crime: 999,999 fake points via UDP ports 6000-6099.
Hates: being dropped.

REWARD: SAMOSAS FOR A MONTH

WANTED



THE PINGER

ping_pong_king

Crime: pinging 10.0.0.1 so often it feels pinged.
Hates: token buckets.

REWARD: ONE CUP OF CHAI

EXHIBIT 3 · THE NOTICE BOARD

PINNED BY THE COUNTER

PIXEL ARCADE · HOUSE RULES
1. No outside chai. We have chai.
2. Samosas are not keyboards.
3. The router is not a footrest.
4. Whoever crashes the server buys the next round.
5. Saturday tournament: 7 pm. Final at midnight.
6. Rage-quitting is allowed. Rage-unplugging is not.

EXHIBIT 4 · THE GROUP CHAT

STAFF CHAT · THE NIGHT IT ALL WENT WRONG

#staff · Saturday 11:47 pm
Owner: the score forger is #1 AGAIN
Rohan: impossible. the bouncer is 100% working
Owner: he has 999,999 points. the game maxes at 50,000
Rohan: ...99% working
Rohan: ok i have exams. good luck. bye
Kabir: i can fix it, i watched one video
Owner: no.
Owner: i'm hiring professionals

EXHIBIT 5 · THE OLD DASHBOARD

BOUNCER v1 · RETIRED · last screenshot

SAMOSAS SOLD	CHEATERS CAUGHT	SERVER MOOD	CHAI LEFT
412	0	SAD	10%

TONIGHT'S LEADERBOARD

1. xX_N00bSlayer_Xx	999,999	(suspicious)
2. chai_wala_99	4,210	
3. SamosaSensei	3,990	
4. rohan (admin)	12	(studying)

status: the bouncer is doing its best

THE RULES FILE

Plain text, one rule per line. `#` starts a comment; blank lines are ignored; words are separated by spaces or tabs. Line numbers count from 1 and include comments and blank lines.

EXAMPLE: PIXEL-ARCADE.RULES

```
# pixel-arcade.rules
default pass
fragments drop
set cheaters 10.0.0.44 10.0.0.45 10.13.37.0/24 2001:db8:bad::/48
set staff    192.168.50.0/24 2001:db8:50::/48
portscan 20 ports in 5s jail 60

drop   src @cheaters log                    # the usual suspects
jail 120 proto udp dport 6000-6099 log      # fake high scores: 2 minutes out
pass   src @staff                           # staff walk straight in
bounce proto udp dport 6666                 # the "secret" cheat port
limit 3/s burst 3  proto icmp icmp-type 8   # pings: 3 per second each
drop   proto tcp dport 23 vlan 100          # no telnet on VLAN 100
limit 500/s burst 1000  proto udp dport 7777 dst 10.0.0.1
honeypot src 10.66.0.0/16                   # let them think they got in
limit 3/s burst 3  proto icmp6 icmp-type 128
```

LINE	MEANING
default <pass|drop>	Verdict when no rule matches. Optional, at most once. Default: pass.
fragments <pass|drop>	Verdict for non-first IPv4 fragments (offset ≠ 0) that no rule has matched. Optional, at most once. Default: drop.
pass [matches]	Let the frame through.
drop [matches]	Throw the frame away.
limit <rate>/s burst <n> [matches]	Rate-limit per source address (see the rate limiter). rate 1–1,000,000, burst 1–1,000,000.
jail <seconds> [matches]	Drop the frame and jail its source address for 1–3,600 seconds (see jail).
bounce [matches]	Send the frame straight back out of the interface it arrived on (see bounce).
honeypot [matches]	Divert the frame, unchanged, to the honeypot interface (see the honeypot).
set <name> [prefix ...]	Declare an address set, or add prefixes to it if the name already exists. Name: 1–32 of a–z, 0–9 and _. Up to 16 sets and 10,000 prefixes in all, IPv4 and IPv6 mixed freely. A set may be empty.
portscan <n> ports in <s>s jail <t>	Turn on the port-scan detector. n 2–1,000, s 1–60, t 1–3,600. Optional, at most once.

MATCH	TRUE WHEN
proto <udp|tcp|icmp|icmp6|any>	The upper-layer protocol is 17, 6, 1 or 58 (any: always).

MATCH	TRUE WHEN
family ⟨ipv4|ipv6|any⟩	The frame is IPv4, IPv6, or either. Default: any.
src ⟨prefix⟩	The source address is inside the prefix: a.b.c.d[/0–32] or an IPv6 prefix such as 2001:db8::/32 (/0–128). No length means a single address. An IPv4 prefix never matches an IPv6 frame, and the other way round.
dst ⟨a.b.c.d[/len]⟩	Same, for the destination address.
src @⟨set⟩ / dst @⟨set⟩	The address is inside any prefix of the set. The set must be declared somewhere in the file.
sport ⟨p | p-q⟩	UDP or TCP source port is p, or between p and q inclusive.
dport ⟨p | p-q⟩	Same, for the destination port.
icmp-type ⟨n⟩	ICMP (IPv4) or ICMPv6 type is n (0–255).
vlan ⟨id | none | any⟩	The frame's 802.1Q VLAN id is id (0–4095), the frame is untagged (none), or either (any). Default: any.
log	Not a condition and takes no value: every frame this rule decides is written to the event log.

- All matches on a line must be true (AND). Each match may appear at most once per line, in any order.
- Rules are checked top to bottom; the first matching rule decides. A rule with no matches matches every IPv4 frame.
- sport and dport only match UDP or TCP frames that carry ports: unfragmented frames and first fragments. icmp-type only matches ICMP and ICMPv6 frames that are not non-first fragments. A rule that uses them never matches anything else.
- Up to 64 rules per file (default, fragments, set and portscan lines are not rules). Any line that breaks this grammar makes the whole file invalid.

WHAT EVERY FRAME GOES THROUGH

STEP	CHECK	RESULT
1	Frame shorter than 14 bytes	PASS, counted as other
2	EtherType 0x8100: one 802.1Q tag. The real EtherType is at bytes 16–17. Frame shorter than 18 bytes, or a second tag (0x8100 or 0x88a8) inside	Tag too short or doubled: PASS, counted as other
3	Real EtherType is neither 0x0800 (IPv4) nor 0x86DD (IPv6): ARP and anything else	PASS, counted as other
4	Malformed IPv4: version ≠ 4, IHL ⟨ 5, or the IPv4 header (IHL × 4 bytes, options included) runs past the end of the frame. Malformed IPv6: see IPv6	DROP, counted as malformed
5	Frame that is not a non-first fragment, whose UDP (8 bytes), TCP (20 bytes), ICMP or ICMPv6 (8 bytes) header runs past the end of the frame	DROP, counted as malformed
6	The source address is in jail	DROP, counted as jailed
7	Port-scan detector, if the file has a portscan line: UDP and TCP frames that carry ports	If this frame jails its source: DROP, counted as jailed

STEP	CHECK	RESULT
8	Everything else	Through the rules: first match, then fragments (for non-first fragments), then default

- Your program must not trust the IPv4 total-length field, the IPv6 payload-length field or any checksum. Trailing bytes after the IP packet (Ethernet padding) are normal and must be ignored.
- Header offsets move: a VLAN tag adds 4 bytes, and IPv4 options add up to 40 more. Rohan learned this the hard way.
- A non-first fragment (fragment offset ≠ 0) carries no transport header, so step 5 does not apply to it.

THE RATE LIMITER

- Each limit rule keeps its own token bucket per source IPv4 address.
- A bucket is created full (burst tokens) the first time a source hits that rule, and refills continuously at rate tokens per second, never above burst.
- A matching frame takes one token and is passed. With less than one token left it is dropped. Either way the rule counts as hit.
- Track at least 65,536 sources per limit rule. When full, forget the source seen least recently.
- Use kernel time (bpf_ktime_get_ns). Judges allow ±1 frame or ±5% per source, whichever is larger.

ADDRESS SETS

- A set is a named list of IPv4 prefixes. `src @name` or `dst @name` matches when the address is inside any of them.
- Prefixes are stored with their host bits cleared, so 10.0.0.5/24 and 10.0.0.0/24 are the same prefix.
- `trafficcop set add` and `trafficcop set del` change a set while running, with no reload. Each exits 0 within 1 second and takes effect for every frame after it exits. Adding a prefix already in the set changes nothing and exits 0. An unknown set, a bad prefix, or deleting a prefix that is not in the set: exit 1.
- A reload replaces every set with the sets in the new file.

JAIL AND PORT SCANS

- A jail rule drops the frame that matches it and jails its source address for the given seconds. While jailed, every IPv4 frame from that address that reaches step 6 is dropped, whatever the rules say.
- Jailing an address that is already jailed keeps whichever release time is later.

- The port-scan detector keeps a window per source address. A window opens with the source's first UDP or TCP frame that carries ports and lasts s seconds; the first such frame after it ends opens a new one. A frame whose destination port is new in the window, and brings the number of distinct destination ports in the window to n or more, jails the source for t seconds and is itself dropped.
- `trafficcop jail` prints the jailed addresses as JSON: [{"src": "10.0.0.66", "seconds_left": 42}], sorted by address, seconds rounded up.
- `trafficcop unjail` releases an address at once: exit 0 if it was jailed, 1 if it was not.
- The jail survives a reload and is emptied by stop. Use kernel time; judges never send a frame within 0.3 s of a release time.

BOUNCE

- A bounce rule sends the frame back out of the interface it arrived on (XDP_TX), with the destination and source MAC addresses swapped and every other byte unchanged.
- A bounced frame is neither passed nor dropped: it is counted as bounced.

THE EVENT LOG

- Every frame decided by a rule marked `log` creates one event.
- `trafficcop log` prints a JSON array of the most recent 1,000 events since start (fewer if there are fewer), oldest first, and exits 0 within 1 second. verdict is PASS, DROP, BOUNCE or HONEYPOT. proto is "udp", "tcp", "icmp", "icmp6" or the protocol number as a string; sport and dport are null when the frame carries no ports.
- No event may be lost at up to 20,000 logged frames per second. The log is kept across reloads (each event keeps the line number it had) and emptied by stop.

OUTPUT OF: TRAFFICCOP LOG

```
[
 {"line": 8, "verdict": "DROP", "src": "10.13.37.4", "dst": "10.0.0.1",
  "proto": "udp", "sport": 40000, "dport": 6010},
 {"line": 8, "verdict": "DROP", "src": "2001:db8:bad::7", "dst": "2001:db8::1",
  "proto": "icmp6", "sport": null, "dport": null}
]
```

IPV6

- Frames with EtherType 0x86DD are IPv6 and go through exactly the same steps and rules as IPv4.
- Extension headers. After the 40-byte header, walk any chain of Hop-by-Hop (0), Routing (43), Fragment (44) and Destination Options (60) headers. A Fragment header is always 8 bytes; the others are (byte 1 + 1) × 8 bytes. The first

next-header value that is none of these is the upper-layer protocol.

- A Fragment header with a non-zero fragment offset makes the frame a non-first fragment, exactly like IPv4.
- **Malformed IPv6:** version ≠ 6, fewer than 40 bytes of header, an extension header that runs past the end of the frame, or more than 8 extension headers. DROP, counted as malformed.
- Rate limits, jail, port scans and top_sources work per full 128-bit source address.
- Wherever trafficcop prints an IPv6 address (stats, jail, log) it uses the shortest standard form (lower case, :: for the longest run of zeros), for example 2001:db8:bad::7. Where addresses are sorted, IPv4 addresses come before IPv6 ones, each in numeric order.
- jail and unjail accept IPv6 addresses; set add and set del accept IPv6 prefixes.

THE HONEYPOT

- `trafficcop start` takes an optional third argument, the name of a **honeypot interface**.
- A **honeypot** rule sends the frame, completely unchanged, out of the honeypot interface instead (XDP_REDIRECT). It never reaches the server.
- A rules file with a honeypot rule is invalid unless trafficcop was started with a honeypot interface: start and reload must exit 1.
- A diverted frame is counted as honeypotted. The judges' honeypot interface is hp0, a veth in the same namespace as eth0.

COUNTERS AND STATS

OUTPUT OF: TRAFFICCOP STATS

```
{
  "passed": 18342, "dropped": 2211, "bounced": 64, "honeypotted": 512,
  "malformed": 7, "other": 120,
  "jailed": 380, "jailed_sources": 2,
  "rules_version": 2, "default_hits": 15013,
  "rules": [
    {"line": 8,  "hits": 1840, "passed": 0,  "dropped": 1840, "bounced": 0, "honeypotted": 0},
    {"line": 9,  "hits": 3,    "passed": 0,  "dropped": 3,    "bounced": 0, "honeypotted": 0},
    {"line": 10, "hits": 96,   "passed": 96, "dropped": 0,    "bounced": 0, "honeypotted": 0},
    {"line": 11, "hits": 64,   "passed": 0,  "dropped": 0,    "bounced": 64, "honeypotted": 0},
    ...
  ],
  "sets": [{"name": "cheaters", "prefixes": 4}, {"name": "staff", "prefixes": 2}],
  "top_sources": [
    {"src": "10.13.37.4", "packets": 1702},
    {"src": "10.0.0.21",  "packets": 990}
  ]
}
```

- passed + dropped + bounced + honeypotted = every frame XDP saw since start, malformed and other included.
- malformed and other count the frames from steps 1–5. jailed counts the frames dropped at steps 6 and 7 (they are also in dropped). jailed_sources is how many addresses are in jail right now.
- rules has one entry per rule, in file order. line is its line number; hits = frames it decided; passed, dropped, bounced and honeypotted split the hits.
- sets lists every set, by name in alphabetical order, with how many prefixes it holds now.
- default_hits = frames decided by default; non-first fragments decided by the fragments line are not in it.
- top_sources: up to 5 IPv4 sources with the most frames that reached step 6, most first; ties by lower address.
- rules_version starts at 1 and goes up by one on each successful reload.
- Counters are exact, even at full speed on many CPUs. They never reset while running, not even on reload. A reload rebuilds the rules list from the new file; its hits start at 0.

THE DASHBOARD

- While running, serve a live page at http://127.0.0.1:8088/ (inside the namespace trafficcop runs in). It must update at least once a second without a manual refresh.

- Show at least: packets per second passed and dropped, rule hits, malformed and other counts, the top sources, who is in jail, and the latest events.
- Also serve /stats.json with exactly the same JSON as `trafficcop stats`.
- Make it look good. Pixel art is encouraged.

THE COMMAND LINE

Your build produces one executable, ./trafficcop. The harness only ever talks to it like this, as root:

COMMAND	MUST
trafficcop start <ifname> <rules file> [<honeypot ifname>]	Validate the rules, load and attach your XDP program to the interface, start any background parts (dashboard included), then exit 0 within 5 seconds, already enforcing. Invalid rules: exit 1, attach nothing.
trafficcop reload <rules file>	Switch to the new rules and exit 0 within 1 second. Invalid rules: exit 1 and keep enforcing the old rules, untouched.
trafficcop stats	Print the stats JSON to standard output and exit 0 within 1 second.
trafficcop stop	Detach the program, remove anything pinned in /sys/fs/bpf, stop background processes and free port 8088, then exit 0 within 5 seconds.
trafficcop set add|del <set> <prefix>	Change an address set live (see address sets). Within 1 second.
trafficcop jail	Print the jailed addresses as JSON (see jail). Within 1 second.
trafficcop unjail <address>	Release an address from jail (see jail). Within 1 second.
trafficcop log	Print the event log as JSON (see the event log). Within 1 second.

- On any failure print one line to standard error. For invalid rules, name the line number.
- Only one trafficcop runs at a time. start while running, or any other command while stopped: exit 1.

HOT RELOAD

- Never detach the program while switching: the interface must stay protected throughout.
- Every frame is judged entirely by the old rules or entirely by the new ones, never a mix.
- Frames that both rule files would pass must never be dropped during a reload, even at full speed.
- Once reload has exited 0, every new frame uses the new rules.

- Rate-limit buckets of a limit rule that appears unchanged at the same position may carry over or restart; either is fine.

YOUR TEST BENCH

You won't get the judges' harness, so build your own. A network namespace and a veth pair give you a private gateway on any Linux machine:

BENCH.SH (RUN AS ROOT)

```
ip netns add arcade
ip link add players type veth peer name eth0 netns arcade
ip link set players up
ip -n arcade link set eth0 up
ip -n arcade link set lo up
# frames sent back with XDP_TX (bounce) only arrive on "players" if it has an
# XDP program of its own: compile any program that just returns XDP_PASS
ip link set dev players xdpdrv obj pass.o sec xdp
# a honeypot next to eth0; frames diverted to hp0 come out of "trap"
ip -n arcade link add hp0 type veth peer name trap netns 1
ip -n arcade link set hp0 up
ip link set trap up
ip link set dev trap xdpdrv obj pass.o sec xdp
# nsenter changes only the network namespace, so /sys/fs/bpf
# stays the normal bpf filesystem (ip netns exec would hide it)
nsenter --net=/run/netns/arcade ./trafficcop start eth0 pixel-arcade.rules hp0
# now send frames into "players" (raw socket, scapy, tcpreplay...)
# and watch what arrives on eth0 inside the namespace
nsenter --net=/run/netns/arcade ./trafficcop stats
nsenter --net=/run/netns/arcade ./trafficcop stop
ip netns del arcade
```

SCORING

Your score depends on both the automatic tests below and a viva, where the judges ask your team about your build. Every team member should be ready to explain any part of it.

AREA	WHAT THE HARNESS CHECKS
Rule engine	Hidden rule files and traffic: prefixes, port ranges, first match, VLANs, IP options, fragments, malformed frames, defaults
Rate limiter	Bursts and steady streams from many sources against several limit rules
Address sets	Hidden sets and traffic around their edges; live set add and del; sets after a reload
Jail and port scans	Jail rules, release times, scanners fast and slow, the jail and unjail commands
Bounce	Bounced frames arrive back with the MACs swapped and nothing else changed
Event log	Thousands of logged frames at full speed: the last 1,000 events, exactly
IPv6	IPv6 rules, prefixes, sets and ICMPv6; extension-header chains, fragments and malformed IPv6
Honeypot	Diverted frames arrive on the honeypot unchanged; files that need a honeypot are refused without one
Counters	Every field of stats after hundreds of thousands of frames at full speed
Hot reload	Reloads during heavy traffic: no good frame lost, new rules in force on exit, bad files rejected
Lifecycle	Start and stop times, clean detach, nothing left in /sys/fs/bpf, the exit codes above
Dashboard	The page loads and updates live; /stats.json matches stats
Speed	Frames per second dropped under a flood, ranked against the other teams

Ties are broken by the Speed result. A build that fails to start scores 0 in every area that needs it running.

GROUND RULES

- Every PASS or DROP decision must be made by your XDP program. User space may only load rules, read counters and serve the dashboard. No iptables, nftables or tc filters.
- Any language for user space: C with libbpf, Go with cilium/ebpf, Rust with Aya, Python with BCC, and so on.
- No existing firewall projects (for example xdp-filter) copied in, whole or in part. Your code, your bugs.
- Judges' machine: Ubuntu 24.04, Linux 6.8, x86-64, 4 CPUs, root. The interface is one end of a veth pair inside a network namespace. trafficcop is run with `nsenter --net=/run/netns/`: only the network namespace changes, and /sys/fs/bpf is the normal bpf filesystem.

WHAT TO HAND IN

- One zip or Git repository, with a build.sh that turns a fresh Ubuntu 24.04 machine into one with a working ./trafficcop. It may install apt packages; list them in the README.
- A README: how it works, how you tested it, and one screenshot of the dashboard.
- Your own test bench and any rule files you used to test.
- Deadline: 13 November 2026, 11:59 p.m. IST.

ONE MORE THING

The owner's last word: "I don't care how clever it is. I care that the cheaters stay out, the players stay in, and the samosas stay warm." Good luck.

REFERENCE CARD · HEADERS

HEADER	BYTE OFFSETS (FROM THE START OF THAT HEADER)
Ethernet (14)	0–5 destination MAC · 6–11 source MAC · 12–13 EtherType: 0x0800 IPv4, 0x86DD IPv6, 0x0806 ARP, 0x8100 802.1Q tag
802.1Q tag	Inserted after the source MAC: 12–13 = 0x8100, 14–15 = TCI (VLAN id in the low 12 bits), 16–17 = real EtherType
IPv4 (20–60)	0 version (high nibble) / IHL in 4-byte words (low nibble) · 2–3 total length · 4–5 id · 6–7 flags (0x2000 = more fragments) + fragment offset (low 13 bits) · 8 TTL · 9 protocol: 1 ICMP, 6 TCP, 17 UDP · 10–11 checksum · 12–15 source · 16–19 destination · 20… options, if IHL > 5
UDP (8)	0–1 source port · 2–3 destination port · 4–5 length · 6–7 checksum
TCP (20+)	0–1 source port · 2–3 destination port · 4–7 seq · 8–11 ack · 12 data offset · 13 flags · 14–15 window
ICMP (8)	0 type: 8 echo request, 0 echo reply, 3 unreachable · 1 code · 2–3 checksum · 4–7 rest of header

All multi-byte header fields are big-endian on the wire. x86-64 is little-endian.

XDP IN ONE BOX

- Your program receives struct xdp_md: data and data_end bound the frame. Every read must be proven to lie inside them, or the verifier rejects the program.
- Return XDP_PASS (2) to let a frame continue into the kernel, XDP_DROP (1) to throw it away.
- Useful map types: ARRAY and PERCPU_ARRAY for counters, HASH and LRU_HASH for per-source state, LPM_TRIE for prefix lookups, RINGBUF for streaming events to user space.
- XDP_TX (3) sends a frame back out of the interface it arrived on; bpf_redirect() with XDP_REDIRECT (4) sends it out of another interface.
- The verifier caps how much work it does per program (1 million instructions examined). Sixty-four rules with many checks each can hit that cap, so watch it as your program grows; the bpf_loop helper can help.
- Useful helpers: bpf_map_lookup_elem, bpf_map_update_elem, bpf_ktime_get_ns. Per-CPU maps or atomic adds keep counters exact.
- Bounded loops are allowed (Linux 5.3+). The verifier still has to prove they end.